ASSIGNMENT 7

Laasya Sri.K 2402MC12

Q1. User-Level Threads

Design and data structures: The implementation adds clone() and join() system-call interfaces. clone() creates a thread-like kernel process that shares the caller's user address space while using its own kernel stack and a supplied user stack. threadtest.c creates three threads, each performs 100,000 increments of the shared volatile global counter. The worker deliberately separates the read and write with a delay to make lost updates more likely.

Problem and handling: The counter is shared, but the read–modify–write sequence is not atomic. Two threads can read the same old value and both write the same incremented value. This test intentionally exposes the race; the correction is demonstrated in Q2 by protecting the increment with a semaphore.

Q2. Semaphores and Producer–Consumer

Design and data structures: Kernel semaphore operations are exposed through sem_init(), sem_wait(), and sem_signal(). The test uses a fixed semaphore array and xv6 sleep()/wakeup() for waiting processes. Semaphore ID 0 protects the counter; IDs 1, 2, and 3 represent empty slots, full slots, and the mutex. The producer–consumer buffer is a circular array of size 5 with in/out indices.

Comparison	**Unsynchronized Q1**	**Synchronized Q2**
Threads / work	3 × 100,000 increments	3 × 100,000 increments
Counter result	106,242 observed	300,000 observed
Interpretation	Lost updates due to race	Mutex semaphore protects critical section

Problem and resolution: The shared counter was incorrect without mutual exclusion. Wrapping counter++ in sem_wait(0)/sem_signal(0) makes the critical section mutually exclusive and produced the expected 300,000. For the bounded buffer, empty/full counting semaphores prevent inserting into a full buffer or removing from an empty one, while the mutex protects buffer indices and contents.

Q3. Deadlock Detection and Banker's Algorithm

Design and data structures: deadlock.c represents each process's Allocation and Request/Need as fixed two-dimensional arrays and resource availability as Available. The detection pass copies Available into Work, marks processes unfinished in Finish[], and repeatedly simulates any process whose outstanding Request can be met; its allocation is returned to Work. Processes remaining unfinished are reported as deadlocked. The Banker safety check computes Need = Max − Allocation and searches for a safe sequence. Resource requests are tentatively applied and rolled back if they cannot be granted safely.

Test case in supplied output/source	**Observed result**	**Interpretation**
1. Detection: circular resource wait	P0, P1, and P2 reported deadlocked	Deadlock detected for the supplied allocation/request matrix.
2. Detection: a process can finish	P2, then P1, then P0 finish; "No deadlock detected"	Returned resources allow all simulated processes to finish.
3. Banker initial state	Safe sequence printed; system reported SAFE	Initial allocation/need/available state is safe.
4. Safe request: P0 requests (0,0,1)	Request GRANTED; resulting state is safe	Tentative allocation passes the safety check.
5. Second request: P0 requests (0,0,2)	Denied for insufficient Available resources	This request was rejected before safety testing; it does not demonstrate a granted request that makes the state unsafe.
Real circular-wait demonstration (supplied Q3 screenshot)	A holds A and waits for B; B holds B and waits for A; A → B → A	Circular wait is demonstrated and both child processes are cleaned up.

Problem / limitation and resolution: The algorithm must distinguish an unsafe state from a request that is simply too large for the currently available resources. In the supplied source, the second Banker request is rejected at the Available check, so it does not establish the assignment's requested "becomes unsafe after a request" test. To complete that requirement, add a request that is within both Need and Available but fails the safety test, then verify that the tentative changes are rolled back. The real-process screenshot demonstrates circular wait separately from the matrix-based detector.

Q4. Priority-Based Preemptive Scheduling with Aging

Design and data structures: The process structure includes a priority and aging/wait tracking. set_priority() sets a process priority and get_priority() reads it. The test forks four CPU-bound children with initial priorities 2, 5, 10, and 20 (smaller number means higher priority). A READY semaphore ensures all children are prepared before a START semaphore releases them to compete. The scheduler selects the highest-priority runnable process; aging reduces the priority number of a process that has waited beyond the configured threshold.

Process (PID in screenshot)	**Initial priority**	**Observed final priority**	**Completion order**
P1 (PID 4)	2	2	1st
P2 (PID 5)	5	5	2nd
P3 (PID 6)	10	9	3rd
P4 (PID 7)	20	19	4th

Problem and handling: Priority alone can starve low-priority work. Aging addresses this by improving the priority of runnable processes that wait too long. PID 6 changes from priority 10 to 9 and PID 7 from 20 to 19, and all four children complete.Observed order is consistent with the initial priorities; it does not by itself quantify elapsed completion time.

Overall Summary

Part	Evidence in supplied files	Status / remaining check
Q1–Q2	Counter 106,242 without lock; 300,000 with semaphore; producer/consumer 1–10	Before/after race comparison demonstrated.
Q3	Deadlock matrix, no-deadlock matrix, Banker safe sequence/request, real circular wait	Add a request that passes availability but fails safety to meet the unsafe-after-request case.
Q4	Completion order 2 → 5 → 10 → 20; priorities 10→9 and 20→19	Numeric completion times are not printed; measure ticks if required for a true completion-time graph.